Smart Parking System - Software Design & Technical Documentation

1. Executive Summary

Project Title: Smart Parking System / Park n Go

The Smart Parking System is a full-stack web application for discovering parking locations, viewing slot availability, reserving parking slots, processing payments, and managing parking operations through an admin dashboard.

The project contains:
* server: Express/MongoDB backend API
* user-dashboard: user-facing React/Vite app
* admin-dashboard: admin-facing React/Vite app
* shared-auth: shared auth/session utilities
* client: combined or legacy frontend copy

Key capabilities include OTP login, Google login, admin login, slot locking, booking confirmation, Razorpay payment verification, parking-location management, slot bulk creation, reports, users, vehicles, and payments management.

2. Project Overview

The system supports two main user groups: parking users and administrators. Parking users search locations, select slots, book parking, pay, and view history. Admins manage cities, pincodes, areas, locations, slots, bookings, users, vehicles, payments, and reports.

A real-world user can search nearby parking, choose a slot, reserve it temporarily, complete payment, and receive a confirmed booking. Admins maintain the service area and parking inventory.

Primary business value:
* Reduces time spent finding parking
* Prevents duplicate slot reservations
* Provides centralized admin control
* Maintains booking/payment history
* Enables structured parking inventory management

3. System Architecture

High-level architecture diagram:

```
‘‘‘mermaid
flowchart LR
  User[User Browser] --> UserApp[user-dashboard React/Vite]
  Admin[Admin Browser] --> AdminApp[admin-dashboard React/Vite]
  UserApp --> API[Express REST API]
  AdminApp --> API
  API --> Mongo[(MongoDB)]
  API --> Twilio[Twilio SMS OTP]
  API --> Firebase[Firebase Admin]
  UserApp --> FirebaseClient[Firebase Web Auth]
  API --> Razorpay[Razorpay Test API]
‘‘‘
```

The backend starts from server/server.js, loads environment variables, connects to MongoDB

through server/config/database.js, configures CORS, mounts Razorpay routes, and mounts the main /api router from server/routes/index.js.

Component architecture:

```
```mermaid
flowchart TB
  Routes[server/routes] --> Middleware[server/middleware/auth.js]
  Routes --> Controllers[server/controllers]
  Controllers --> Models[server/models]
  Controllers --> Services[server/services]
  Models --> MongoDB[(MongoDB)]
```
```

Request lifecycle:
1. Frontend calls an API wrapper from src/services/api.js.
2. Shared Axios client attaches Authorization: Bearer <token>.
3. Express route validates input.
4. Auth middleware verifies JWT where required.
5. Controller executes business logic.
6. Mongoose model reads/writes MongoDB.
7. JSON response is returned to the frontend.

Sequence diagram:

```
```mermaid
sequenceDiagram
  participant Browser
  participant React
  participant API as Express API
  participant Mongo as MongoDB
  participant Gateway as External Provider
  Browser->>React: User action
  React->>API: Axios request
  API->>API: Validate request and JWT
  API->>Mongo: Query/update models
  API->>Gateway: Optional SMS/Firebase/Razorpay call
  Gateway-->>API: Provider response
  Mongo-->>API: Database result
  API-->>React: JSON response
  React-->>Browser: Updated UI
```
```

4. Technology Stack

Backend runtime: Node.js. Executes the Express API server.
Backend framework: Express ^4.18.2. Provides routing and middleware.
Database: MongoDB with Mongoose ^7.5.0. Provides document storage, schemas, validation, indexes, and model methods.
Authentication: jsonwebtoken ^9.0.2 and bcryptjs ^2.4.3. JWTs are used for stateless sessions; bcrypt hashes passwords.
Validation: express-validator ^7.0.1. Performs route-level request validation.
SMS: Twilio ^5.5.3. Sends OTP messages.
Firebase: firebase-admin ^13.10.0 and firebase ^12.1.0. Used for Google authentication verification.
Payments: Razorpay ^2.9.6. Creates orders and verifies payment signatures in test mode.
Frontend: React and React DOM ^18.2.0. Builds user and admin SPAs.
Build tool: Vite ^7.3.1. Provides development server and frontend build pipeline.
Routing: react-router-dom ^6.15.0. Handles SPA navigation.
HTTP client: Axios ^1.5.0. Performs API calls with auth interceptors.

Styling: Tailwind CSS ^3.3.3, PostCSS, Autoprefixer. Provides utility-first CSS.
Forms: react-hook-form, yup, @hookform/resolvers. Handles form state and validation.
UI helpers: lucide-react, react-icons, react-toastify, SweetAlert2. Provides icons, notifications, and dialogs.
Charts: Recharts ^2.7.2. Supports admin analytics.
Maps: Leaflet, React Leaflet, Google Maps wrapper. Supports parking location/map features.
CSV import: PapaParse ^5.5.3. Supports admin data import parsing.

5. Folder Structure Explanation

server/server.js: Backend entrypoint.
server/routes: REST route definitions.
server/controllers: Request handlers and business logic.
server/models: Mongoose schemas.
server/middleware: Auth, authorization, and error middleware.
server/config: Database and Firebase Admin setup.
server/services: Email, OTP, Firebase user sync, Twilio helpers.
server/utils: Common helper utilities.
admin-dashboard/src/admin/pages: Admin dashboard pages.
admin-dashboard/src/components: Shared admin UI components.
admin-dashboard/src/context: Admin auth context.
admin-dashboard/src/services/api.js: Admin API client.
user-dashboard/src/user/pages: User-facing pages.
user-dashboard/src/user/components: Parking map and payment components.
user-dashboard/src/context: User auth context.
user-dashboard/src/services/api.js: User API client.
shared-auth: Shared auth storage, service, and Axios client.
docs: Existing project documentation.
client: Combined or legacy frontend copy.

6. Functional Modules

Authentication: Implemented through routes/auth.js, authController.js, authPlaceholderController.js, and middleware/auth.js. Supports OTP login/signup, Google login, admin login, and JWT profile lookup.

Location hierarchy: Implemented through cities, pincodes, areas, and locations routes/controllers/models. Provides admin CRUD for geographic hierarchy.

Parking slots: Implemented through slots.js, slotController.js, and ParkingSlot.js. Supports slot CRUD, bulk slot creation, status, lock fields, and pricing.

Booking: Implemented through bookings.js, bookingController.js, and Booking.js. Supports smart booking, slot locking, cancellation, extension, check-in, and check-out.

Payment: Implemented through payments.js, razorpayPayments.js, paymentController.js, and razorpayController.js. Supports payment initiation, Razorpay verification, simulation, payment history, and receipts.

Vehicles: Implemented through vehicles.js, vehicleController.js, and Vehicle.js. Supports user vehicle CRUD.

Users: Implemented through users.js, userController.js, and User.js. Supports profile and admin user management.

Reports: Implemented through reports.js, reportController.js, and Report.js. Supports dashboard

stats and report records.

Imports: Implemented through imports.js and importController.js. Supports admin data import.

Notifications and activity logs: Implemented through Notification.js and ActivityLog.js. Persist notifications and audit events.

7. User Workflows

Registration/Login workflow:

```
‘‘‘mermaid
flowchart TD
  Start[Start] --> Method{Choose auth method}
  Method --> Phone[Enter phone]
  Phone --> OTP[Send OTP]
  OTP --> Verify[Verify OTP]
  Verify --> JWT[Issue JWT]
  Method --> Google[Google sign-in]
  Google --> FirebaseToken[Send Firebase ID token]
  FirebaseToken --> ServerVerify[Verify with Firebase Admin]
  ServerVerify --> JWT
  JWT --> Store[Store token and user in localStorage]
‘‘‘
```

Parking discovery workflow:

```
‘‘‘mermaid
flowchart TD
  Search[Search page] --> Nearby[GET /api/locations/nearby]
  Nearby --> Match[Match by radius, city, area, or pincode]
  Match --> Count[Count available slots]
  Count --> Blueprint[GET /api/locations/:id/slots]
  Blueprint --> SlotMap[Render floor-wise slot map]
‘‘‘
```

Slot reservation and payment workflow:

```
‘‘‘mermaid
flowchart TD
  Select[Select slot] --> Create[POST /api/bookings/create]
  Create --> Validate[Validate slot, location, vehicle, time]
  Validate --> Conflict[Check overlapping bookings]
  Conflict --> Lock[Lock slot for 5 minutes]
  Lock --> Initiate[POST /api/payments/initiate]
  Initiate --> Verify[POST /api/payments/verify]
  Verify --> Success{Payment valid?}
  Success -->|Yes| Confirm[Booking confirmed and slot occupied]
  Success -->|No| Release[Booking cancelled and slot released]
‘‘‘
```

Admin workflow:

```
‘‘‘mermaid
flowchart TD
  Login[Admin login] --> Dashboard[Dashboard]
  Dashboard --> Master[Manage cities, pincodes, areas, locations]
  Dashboard --> Slots[Manage slots and bulk creation]
  Dashboard --> Ops[Manage users, vehicles, bookings, payments]
  Dashboard --> Reports[View reports and stats]
‘‘‘
```

8. Database Design

Entity relationship diagram:

```
‘‘‘mermaid
```

```
erDiagram
  USER ||--o{ VEHICLE : owns
  USER ||--o{ BOOKING : creates
  USER ||--o{ PAYMENT : makes
  USER ||--o{ NOTIFICATION : receives
  ADMIN ||--o{ REPORT : generates
  CITY ||--o{ PINCODE : contains
  CITY ||--o{ AREA : contains
  CITY ||--o{ LOCATION : contains
  PINCODE ||--o{ AREA : contains
  PINCODE ||--o{ LOCATION : contains
  AREA ||--o{ LOCATION : contains
  LOCATION ||--o{ PARKING_SLOT : has
  PARKING_SLOT ||--o{ BOOKING : reserved_by
  VEHICLE ||--o{ BOOKING : used_for
  BOOKING ||--o| PAYMENT : has
```

Models and important fields:
User: name, email, phone, password, firebaseUid, role, isActive, authProviders.
Admin: name, email, password, role, permissions, isActive.
City: name, state, status.
Pincode: pincode, cityId, status.
Area: name, pincodeId, cityId, status.
Location: name, lat, lng, areaId, pincodeId, cityId, floors.
ParkingSlot: slotNumber, locationId, city, area, pincode, vehicleType, slotType, price, status, lockExpiresAt.
Booking: user, parkingSlot, vehicle, bookingReference, status, startTime, endTime, pricing, paymentLock.
Payment: user, booking, amount, method, gateway, status, Razorpay IDs, verification.
Vehicle: owner, licensePlate, make, model, year, color, vehicleType.
Notification: user, type, title, message, channels, status.
ActivityLog: user, admin, action, resource, description, severity.
Report: title, type, parameters, data, summary, generatedBy.

Key constraints include unique user email/phone, unique admin email, unique city/state, unique pincode per city, unique area per city/pincode, and ObjectId relationships across models.

9. API Documentation

Base API path: /api

Health:
GET /health - server/database health, public.

Authentication:
POST /api/auth/send-otp - send OTP, public.
POST /api/auth/verify-otp - verify OTP, public.
POST /api/auth/signup - signup with OTP, public.
POST /api/auth/login - start OTP login, public.
POST /api/auth/login/verify - verify login OTP, public.
POST /api/auth/google - Google auth, public.
GET /api/auth/profile - current profile, JWT required.
POST /api/auth/admin/login - admin login, public.

Locations:
GET /api/locations/nearby - nearby locations, public.
GET /api/locations/:id/slots - slot blueprint, public.
GET /api/locations/public - public locations, public.

GET/POST /api/locations - list/create locations, admin.
GET/PUT/DELETE /api/locations/:id - manage location, admin.

Cities, Pincodes, Areas:
GET/POST /api/cities - list/create cities, admin.
GET/PUT/DELETE /api/cities/:id - manage city, admin.
GET/POST /api/pincodes - list/create pincodes, admin.
GET/PUT/DELETE /api/pincodes/:id - manage pincode, admin.
GET/POST /api/areas - list/create areas, admin.
GET/PUT/DELETE /api/areas/:id - manage area, admin.

Slots:
GET /api/slots - list slots, public.
GET /api/slots/available - available slot endpoint, public.
GET /api/slots/:id - get slot, public.
POST /api/slots - create slot, admin.
POST /api/slots/bulk - bulk create slots, admin.
PUT/DELETE /api/slots/:id - update/delete slot, admin.

Bookings:
GET /api/bookings - list bookings, JWT required.
GET /api/bookings/me - current user bookings, JWT required.
POST /api/bookings/create - smart booking, JWT required.
POST /api/bookings - legacy booking, JWT required.
GET/PUT /api/bookings/:id - get/update booking, owner/admin.
PUT /api/bookings/:id/cancel - cancel booking, owner/admin.
PUT /api/bookings/:id/extend - extend booking, owner/admin.
POST /api/bookings/:id/checkin - check in, owner.
POST /api/bookings/:id/checkout - check out, owner.

Payments:
GET /api/payments - list payments, JWT required.
POST /api/payments/initiate - initiate payment, JWT required.
POST /api/payments/verify - verify payment, JWT required.
GET /api/payments/:id - payment details, owner/admin.

Vehicles:
GET/POST /api/vehicles - user vehicles, JWT required.
GET/PUT/DELETE /api/vehicles/:id - manage vehicle, JWT required.

Users:
GET /api/users - list users, admin.
GET/PUT /api/users/:id - get/update user, JWT required.
DELETE /api/users/:id - delete user, admin.

Reports and Imports:
GET /api/reports/dashboard - dashboard stats, admin.
GET/POST /api/reports - reports, admin.
GET/DELETE /api/reports/:id - report details/delete, admin.
POST /api/imports/:type - import data, admin.

Common success response: { "success": true, "message": "Optional message", "data": {} }
Common error response: { "success": false, "message": "Error message" }

10. Authentication & Security

Authentication is JWT-based. Tokens are signed with JWT_SECRET and expire using JWT_EXPIRE or a default of seven days.

Security mechanisms found:
* Password hashing with bcrypt pre-save hooks.
* JWT verification in protect middleware.
* Role checks through authorize middleware.
* Admin/superadmin protected routes.
* Input validation through express-validator.
* CORS origin configuration from CLIENT_URL.
* Razorpay HMAC signature verification.
* Inactive user blocking.
* Activity logging for important events.

Session handling:
* shared-auth/authStorage.js stores token and user in localStorage.
* shared-auth/apiClient.js injects Bearer tokens.
* HTTP 401 clears local storage and redirects.

Security risks and recommendations:
* OTP is stored in memory. Use Redis or database storage for production.
* Some Razorpay routes are public. Add JWT and ownership checks.
* JWT is stored in localStorage. Harden CSP or use httpOnly cookies.
* Logout route is GET and not protected. Use protected POST logout.
* Firebase web config is hardcoded. Move it to environment variables.

11. Core Business Logic

Smart Booking Logic:
1. Receive locationId, parkingSlotId, vehicleType, start time, and duration.
2. Validate location and slot.
3. Resolve user vehicle or create placeholder vehicle.
4. Release expired slot lock if needed.
5. Reject reserved, occupied, inactive, incompatible, or unpriced slot.
6. Check conflicting confirmed/active bookings.
7. Reject slot locked by another user.
8. Calculate subtotal, 18 percent tax, and final amount.
9. Create pending booking with payment lock.
10. Mark slot as locked for five minutes.
11. Return booking and lock expiry.

Conflict Detection:
The Booking.findConflictingBookings method checks same parking slot, active/confirmed statuses, and interval overlap using existing.startTime < requestedEnd and existing.endTime > requestedStart.

Payment Verification:
On success, payment becomes completed, booking becomes confirmed, booking payment status becomes paid, slot lock is cleared, and slot status becomes occupied. On failure, payment becomes failed, booking becomes cancelled, and slot is released to available.

Slot Bulk Creation:
slotController.createBulkSlots accepts prefixes, ranges, floor, vehicle type, slot type, and price. It generates slot numbers, skips existing duplicates, and inserts remaining slots.

12. Frontend Documentation

User Dashboard Routes:
/ - Home
/login - Login
/signup - Signup
/search - Search
/parking/:parkingLotId - Parking lot details
/booking - Booking
/history - History
/profile - Profile
/payments - Payments

Important user components:
ParkingLotSlotMap.jsx, HeroParkingMap.jsx, PaymentButton.jsx, Receipt.jsx, ReceiptTicket.jsx, PaymentMethodSelector.jsx, PaymentSuccessPanel.jsx.

Admin Dashboard Routes:
/admin/login - Login
/admin/dashboard - Dashboard
/admin/parking-slots - Parking slots
/admin/bookings - Bookings
/admin/users - Users
/admin/vehicles - Vehicles
/admin/payments - Payments
/admin/city - City
/admin/pincode - Pincode
/admin/area - Area
/admin/location - Location
/admin/reports - Reports
/admin/settings - Settings
/admin/profile - Profile

Frontend state is handled with React Context, not Redux. Routing uses React Router. API access uses Axios wrappers with shared auth storage and interceptors.

13. Backend Documentation

Backend layers:
* server/server.js: entrypoint.
* server/routes/*.js: routing.
* server/controllers/*.js: business logic.
* server/models/*.js: database schemas.
* server/middleware/auth.js and errorHandler.js: auth and errors.
* server/config/database.js and firebaseAdmin.js: configuration.
* server/services/*.js: Firebase user sync, email templates, OTP throttling, Twilio Verify helpers.

The backend uses controller-level business logic rather than a repository pattern. Mongoose models include validation, indexes, virtuals, hooks, and helper methods.

14. Deployment Process

Environment variables:

NODE_ENV, PORT, CLIENT_URL, MONGODB_URI, MONGODB_CONNECT_ON_START, MONGODB_SERVER_SELECTION_TIMEOUT_MS, ALLOW_SERVER_WITHOUT_DB, JWT_SECRET, JWT_EXPIRE, ALLOW_DEV_ADMIN_LOGIN, DEV_ADMIN_EMAIL, DEV_ADMIN_PASSWORD, TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_PHONE_NUMBER, TWILIO_VERIFY_SERVICE_SID, RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, FIREBASE_SERVICE_ACCOUNT_JSON, FIREBASE_ADMIN_PROJECT_ID, FIREBASE_ADMIN_CLIENT_EMAIL, FIREBASE_ADMIN_PRIVATE_KEY, VITE_API_BASE_URL, VITE_AUTH_REDIRECT_PATH.

Run/build commands:
server: npm run dev, npm start.
admin-dashboard: npm run dev, npm run build, npm run preview.
user-dashboard: npm run dev, npm run build, npm run preview.
client: npm run dev, npm run build, npm run preview.

Recommended production layout:
Static hosting/CDN serves user and admin SPAs. Both call the Node Express API. The API connects to MongoDB Atlas and external services such as Twilio, Razorpay, and Firebase.

15. Testing

No formal unit-test or integration-test framework was found in package scripts. Existing manual backend helpers include server/testApi.js, server/testTwilio.js, server/seed.js, and server/cleanup.js.

Recommended testing additions:
* Unit tests for booking conflict detection.
* Integration tests for smart booking and payment verification.
* Admin CRUD API tests.
* Frontend smoke tests for login, booking, and slot creation.
* CI pipeline running lint, build, and tests.

16. Design Decisions

Separate user/admin dashboards provide cleaner role-specific UI.
Shared auth utilities reduce duplicated token and API-client logic.
Slot lock before payment reduces double booking risk.
Snapshot data in bookings/payments keeps receipts stable after master-data edits.
REST-only updates simplify deployment but reduce real-time accuracy.
Razorpay test-mode restriction is safer for project/demo mode.

17. Future Improvements

* Add WebSocket or SSE live slot updates.
* Move OTP storage/rate limiting to Redis.
* Add background cleanup for expired slot locks.
* Protect all Razorpay user routes.
* Implement true time-based filtering in /api/slots/available.
* Add complete OpenAPI specification.
* Add automated tests and CI.
* Move Firebase client config to environment variables.
* Add fine-grained admin permissions.
* Add production-grade logging and monitoring.
* Add database transactions for booking/payment/slot state changes.

18. Conclusion

The Smart Parking System is a complete full-stack parking management application built around React dashboards, an Express API, MongoDB persistence, JWT authentication, OTP/Google login, slot locking, booking management, and payment verification.

Its strongest implemented workflow is the smart booking flow: users discover locations, inspect slot blueprints, lock a slot, initiate payment, verify payment, and receive a confirmed booking while the backend updates slot occupancy state. The project is suitable for a final-year engineering demonstration and can be evolved toward production with stronger testing, real-time updates, hardened payment/auth routes, persistent OTP storage, and operational monitoring.